Assignment 03

SE4095 Social Network Theory and Practice

1 Assignment task

An organization wants to spread an awareness message through its workplace contact network. It can directly promote the message to only five participants. Highly connected participants may reach overlapping audiences, so the five highest-degree participants may not form the best seed set.

Implement a reinforcement-learning agent that selects five participants sequentially. Train an MLP policy and a compulsory Transformer policy using REINFORCE. Compare both policies against Random and Degree selection under identical diffusion conditions.

Learning outcomes

- Analyze network connectivity and centrality; implement information diffusion.
- Define observations, actions, rewards and termination for a budgeted selection task.
- Implement policy-gradient learning and Transformer self-attention.
- Evaluate stochastic outcomes using controlled, reproducible experiments.

2 Dataset and network analysis

. Use SocioPatterns Contacts in a Workplace, collected in an office building in France from June 24 to July 3, 2013. The release contains timestamped face-to-face contacts and anonymized participant department labels.

Official dataset page and downloads

Raw contact list ZIP

Participant department metadata

Use the instructor-supplied static package derived from this release: nodes.csv, edges.csv, departments.csv and dataset_README.md. Preserve its 92 participants and all supplied connections. The source links identify provenance; do not independently regenerate or resample the assignment graph.

Report participant and edge counts, density, connected components, degree centrality, local clustering coefficients, a top-10 degree-centrality table and one readable network visualization. Department labels are allowed only for visualization and final analysis.

3 Fixed dataset interpretation

The shared package must follow these rules so every submission uses the same network.

1. Include every participant in the department metadata, including participants with no retained contacts. Sort original numeric IDs and map them to consecutive IDs 0–91.
2. Aggregate the entire supplied contact period. Connect two participants if at least one recorded contact exists between them.
3. Treat the graph as undirected and unweighted. Remove self-loops and duplicate pairs. Store each edge once with source < target, sorted by source and target.
4. Retain the original ID mapping in nodes.csv. Store department labels separately in departments.csv. Do not use contact duration or timestamps as additional inputs.

The contacts are observed data. Message adoption and the activation probability below are simulation assumptions; they are not measured information-sharing behavior.

4 Information diffusion

Implement the Independent Cascade Model yourself. The five selected participants are seed users. Each newly activated user has one opportunity to activate each inactive neighbor, with independent probability 0.10. Active users stay active. Continue until no new users activate. Final spread includes the seeds.

Required reproducible simulator

1. Represent every undirected edge as two directed arcs, sorted by (source, target).
2. Initialize numpy.random.default_rng(diffusion_seed). Draw one uniform random number for each arc in that order.
3. Mark an arc live when its draw is less than 0.10. Final spread is the number of participants reachable from the seed set through live arcs.
4. Keep live arcs and sampled random numbers private to the simulator. Policies may not inspect them or query hypothetical action outcomes.

Required verification

- An empty seed set produces zero spread.
- At probability 0, only the distinct seeds are active.
- At probability 1, all users reachable from the seeds are active.
- The same graph, seed set and diffusion seed produce the same spread.

Probabilities 0 and 1 are permitted only for verification. All training and evaluation use 0.10. Use the same ordered arc list for every simulation.

5 Reinforcement learning environment

Implement reset(diffusion_seed) and step(action). Use N = 92 participants and a promotion budget of five.

Observation and action mask

Return an N × 5 feature matrix in ascending node-ID order and a Boolean mask identifying unselected participants. Both policies receive exactly these features:

Feature	Definition
Degree centrality	Degree / (N − 1)
Clustering coefficient	Local clustering coefficient on the fixed graph
Selected indicator	1 if previously selected, otherwise 0
Selected neighbor fraction	Selected neighbors / degree; 0 for an isolated node
Remaining budget fraction	Remaining selections / 5

Compute static features once; update selection-related features after every action. Do not include department labels, original IDs as features, live arcs, future spread, or additional node embeddings.

Actions and transitions

- An action is an integer from 0 to 91 that identifies an unselected participant.
- Exactly five distinct participants must be selected per episode.
- Mask selected participants before computing action probabilities.
- Raise an error for an invalid action; do not silently replace it.
- After a valid action, update the selected set, features and action mask.

Reward and termination

Diffusion runs only after the fifth selection. There is no intermediate cascade feedback. The first four rewards are zero; the fifth reward is:

$$R = final\ spread / N$$

Terminate immediately after action five. The total episode reward is R and lies between 0 and 1. Do not introduce penalties, reward shaping or additional objectives.

Interpretation

Explain why this is sequential decision-making: each selection changes the remaining choices and the selected-neighbor features. The goal is a useful combination of users, rather than five independent popularity scores. Training and evaluation use the same graph; only diffusion realizations change.

6 REINFORCE implementation

Implement REINFORCE yourself in PyTorch. Collect 20 complete episodes before each update. Sample actions from the masked categorical policy during training and retain their differentiable log-probabilities until the update.

For episode i, let R_i be its terminal reward. Use the same return at all five selections because the discount factor is 1. Minimize:

$$L(\theta) = -\frac{1}{20}\sum_{i=1}^{20}(R_i - b)\sum_{t=1}^{5} log\,\pi\theta(a_{i,t} \mid s_{i,t})$$

Here, πθ is the policy; a is a chosen user; s is the observation; and b is a scalar running reward baseline. Returns and the baseline must not receive gradients.

Baseline procedure

1. Initialize b = 0 at the start of every training run.

2. Keep b fixed while collecting a batch and calculating its loss. Use only the baseline from previous batches.

3. Perform one optimizer step using the complete 20-episode batch.

4. After the step, update b using the batch mean reward $\bar{R}$:

$$b \leftarrow 0.9b + 0.1\bar{R}$$

Discard trajectories after the update. Explain how the return-weighted log-probabilities change selection behavior and why the baseline can reduce gradient variability.

Fixed training settings

Setting	Required value
Optimizer	Adam
Learning rate	0.001
Discount factor	1.0
Episodes per update	20
Optimizer steps per batch	1
Maximum gradient norm	1.0
Training episodes per run	1,000
Training seeds	7, 21, 42

Use no PPO, critic, replay buffer, entropy bonus, advantage or reward normalization, learning-rate schedule, early stopping or alternative RL algorithm. There are 50 updates per training run.

7 Required neural policies

Train both policies using the same environment, features, REINFORCE code and training settings. Both output one action logit per participant.

MLP policy 8 marks

Apply the same network independently to each participant:

```
5 → Linear(5,64) → ReLU → Linear(64,64)
  → ReLU → Linear(64,1)
```

Mask selected participants and apply softmax across all N participants. Use no dropout or additional layers. This baseline scores each participant from its own five features.

Transformer policy 12 marks

Represent each participant as one token. Apply:

```
N × 5 → Linear(5,64) → Transformer encoder layer
      → Linear(64,1) on each token → masked softmax
```

Setting	Required value
Model dimension	64
Attention heads	4
Encoder layers	1
Feed forward dimension	128
Activation	ReLU
Dropout	0
Normalization	Post layer normalization

- Do not use positional embeddings, node-ID embeddings or causal masks.
- Keep selected participants visible to self-attention. Mask them only at the final action-selection stage.
- Allow attention across all N participants. Do not restrict attention to graph edges.
- Use the default bias terms in linear and Transformer layers. Use no extra final encoder normalization beyond that within the specified layer.

Framework-provided Transformer encoder layers are permitted. Ready-made reinforcement-learning agents and pre-trained models are prohibited.

Required explanation

Explain participant tokens, queries, keys, values, multi-head attention, residual connections and normalization. Explain how attention differs from graph connectivity and how selected-user information can influence later choices. REINFORCE is the learning algorithm; the Transformer is a policy architecture.

8 Controlled evaluation

Compare exactly these four methods:

Method	Selection rule
Random	Uniformly sample five distinct users
Degree	Choose the five highest-degree users
REINFORCE MLP	Sequential selection using a trained MLP
REINFORCE Transformer	Sequential selection using a trained Transformer

Break degree ties using the smaller node ID. During neural evaluation, choose the highest-probability valid action; break exact ties using the smaller ID. Disable gradients and do not update the policy.

Training seeds and checkpoints

At the start of run s, set Python, NumPy and PyTorch seeds to s. For episode e = 0,…,999, use diffusion_seed = 1000000 + 10000 × s + e. Train each architecture with seeds 7, 21 and 42, producing six runs. Use the final checkpoint after episode 1,000.

Evaluation protocol

Use diffusion seeds 900000–900099 inclusive. Evaluate all six checkpoints and the two baselines on those same 100 realizations: exactly 800 evaluation episodes. For Random, use a separate generator with seed diffusion_seed + 10000000 and sample without replacement. Do not tune settings using evaluation results.

Required results

- For each method and training seed: mean spread, population standard deviation of spread, mean normalized reward, and mean selection time excluding diffusion.
- For each learned run: training time including training simulations. Synchronize device operations before timing when applicable.
- For each neural architecture: mean and population standard deviation of its three run-level mean spreads.
- Learning curves using non-overlapping blocks of 20 training episodes, plus a final-spread comparison plot.
- Selected-user table and department coverage for each learned checkpoint and Degree. For Random, report mean department coverage over 100 episodes.
- One network visualization showing the Transformer selection from training seed 7.

Discuss overlapping audiences, department coverage, learning stability and computational cost. Department coverage means the number of distinct recorded departments represented among selected users; it is descriptive only. A policy is not required to outperform Degree to receive full implementation credit.

9 Submission and reproducibility

Submit RollNumber_SNTP_Assignment03.zip containing one outer folder with the same name without .zip. Use this structure:

```
RollNumber_SNTP_Assignment03/
  main.py
  environment.py
  models.py
  reinforce.py
  requirements.txt
  README.md
  report.pdf
  data/
    nodes.csv
    edges.csv
    departments.csv
    dataset_README.md
  results/
    graph_edges.csv
    training.csv
    evaluation.csv
    summary.csv
    checks.json
    checkpoints/
    figures/
```

Run all required experiments using python main.py --output results. Use relative paths and record dependency versions. Include the supplied data package unchanged. Additional helper files are allowed.

Required CSV schemas

graph_edges.csv — source,target. Each undirected pair once, source < target, sorted lexicographically.

training.csv — exactly 6,000 rows. Columns: method, train_seed, episode, final_spread, total_reward. Episodes are numbered 0–999.

evaluation.csv — exactly 800 rows. Columns: method, train_seed, diffusion_seed, selected_nodes, final_spread, total_reward, selection_seconds.

summary.csv — exactly 8 rows. Columns: method, train_seed, spread_mean, spread_std, reward_mean, selection_seconds_mean, training_seconds.

Method names: random, degree, reinforce_mlp, reinforce_transformer. Use train_seed = −1 and training_seconds = 0 for Random and Degree. Encode selected_nodes as a quoted JSON list in selection order. Report time in seconds and rewards on the 0–1 scale. Use population standard deviation throughout.

Checks and checkpoints

checks.json must record pass/fail for the four diffusion checks, duplicate-action rejection, five distinct selected users, reward = spread/N, zero probability for masked actions, and finite training losses and gradients. Save six final checkpoints with parameters and architecture settings.

Report and README

Submit a searchable report of 4–6 pages excluding references: network analysis, RL formulation, simulator, REINFORCE, architectures, results and limitations. The README must give installation and execution instructions, hardware, expected runtime and incomplete requirements.

10 Mandatory limitations

- Use only the supplied 92-participant workplace graph. Preserve all nodes and edges.
- Use only Independent Cascade with activation probability 0.10 for experiments.
- Select exactly five distinct users per episode and use only the specified five input features.
- Implement only the specified MLP and compulsory Transformer policies with REINFORCE.
- Use exactly three training seeds, 1,000 episodes per run, and 100 evaluation diffusion realizations.
- Do not perform hyperparameter searches, introduce additional reward terms, or use pre-training or external datasets.
- Do not inspect hidden live arcs, evaluate hypothetical actions during selection, or access simulator random states.
- Do not implement PPO, GCN, LSTM, recommendation, community detection or link prediction as additional experiments.
- Do not use department labels as inputs, rewards or tuning signals.
- Limit claims to this fixed aggregated contact graph. Static aggregation discards contact timing and duration; simulated spread is not observed message adoption.

11 Marking rubric

Component	Marks
Dataset and network analysis	10
Independent Cascade simulator	15
Reinforcement learning environment	15
REINFORCE implementation	20
MLP and compulsory Transformer policies	20
Controlled evaluation and interpretation	15
Reproducibility and submission	5
Total	100

Assessment rewards correctness, compliance with the protocol, reproducible evidence and interpretation. No minimum spread or improvement over Degree is prescribed. Cite adapted code and external resources. Every student must be able to explain action masking, reward calculation, the policy-gradient update and the Transformer policy.

12 Dataset attribution and supporting references

SocioPatterns Contacts in a Workplace. The original release uses a Creative Commons Public Domain Dedication. Acknowledge SocioPatterns and cite Génois et al., Data on face-to-face contacts in an office building suggest a low-cost vaccination strategy based on community linkers, Network Science 3, 326–347, 2015.

Dataset source and terms

Original workplace study DOI

Supporting RL reading: Williams, Simple statistical gradient-following algorithms for connectionist reinforcement learning, Machine Learning 8, 229–256, 1992. Required preparation: review a short REINFORCE briefing alongside the neural-network slides before implementation.

REINFORCE paper DOI